Here the variable names are the same as the property names on the object, so it's repetitive to write both:

```
highlight = ->
initialize = ->

object =
  highlight: highlight
  initialize: initialize
```

Repetitive

Instead of being repetitive, the object shorthand means you can pack variable values back into an object based on their names:

```
makeCompetition = ->
  find = ->
  color = ->
  highlight = ->
  initialize = -> 'initialized'

  {highlight, initialize}

competition = makeCompetition()
competition.initialize()
# 'initialized'
```

Use object shorthand to export the value of the variable with the name.

The object shorthand also works when specifying parameters. Suppose `makeCompetition` accepts an `options` argument containing the `maxCompetitors` and the `sortOrder`:

```
makeCompetition = (options) ->
  options.max
  options.sort
```

Although an `options` argument saves you from having an unwieldy number of arguments to the function, the end result is often repetitive regardless. You can avoid that by using object destructuring directly in the parameter list. An object in the parameter definition will result in the object properties being destructured to individual named parameters:

```
makeCompetition = ({max, sort}) ->
  {max, sort}
```